

Pre-Processing Decisions

Original row count: 47364

Filtered row count: 39264

Accepted count: 17593

Rejected count: 21671

Text fields used (3): ['comments', 'llm-generated-program', 'llm-generated-university']

Non-text fields used (6): ['GPA', 'GRE', 'GRE V', 'GRE AW', 'Degree', 'US/International']

Full field list (9): ['comments', 'llm-generated-program', 'llm-generated-university', 'GPA', 'GRE', 'GRE V', 'GRE AW', 'Degree', 'US/International']

Preview of cleaned DataFrame:

comments	llm-generated-program	llm-generated-university	GPA	GRE	GRE V	GRE AW	Degree	US/International	label
Unknown	Environmental Economics	Stockholm University	NaN	NaN	NaN	NaN	PhD	International	1
Unknown	Global Health	Meharry Medical College	3.90	NaN	NaN	NaN	PhD	American	1
Applying as a (home student) Oxford undergrad. Didn't receive the offer until super late...!	Engineering Science	University of Oxford	NaN	NaN	NaN	NaN	PhD	Other	1
Duke Master of Science in ME, GPA: 3.85	Mechanical Engineering	Rice University	3.84	NaN	NaN	NaN	PhD	International	1
Unknown	Computer Science	Georgia Institute of Technology	3.90	NaN	NaN	NaN	PhD	American	0

(Preview also prints when running the training program)

While I was previewing the data I noticed some of the comment fields included words that might key the model to be predisposed to answer an affirmative or negative based purely on their presence. I made a small effort to redact some of those words in order to maintain data integrity.

Model Input Template

Program: {program}
University: {university}
Comments: {comments}
Degree: {degree}
Citizenship: {citizenship}
GPA: {gpa}
GRE Quant: {gre_quant}
GRE Verbal: {gre_verbal}
GRE AW: {gre_aw}

Sample 1

Program: Environmental Economics
University: Stockholm University
Comments: Unknown
Degree: PhD
Citizenship: International
GPA: Not reported
GRE Quant: Not reported
GRE Verbal: Not reported
GRE AW: Not reported

Sample 2

Program: Global Health
University: Meharry Medical College
Comments: Unknown
Degree: PhD
Citizenship: American
GPA: 3.9
GRE Quant: Not reported
GRE Verbal: Not reported
GRE AW: Not reported

Sample 3

Program: Engineering Science
University: University of Oxford
Comments: Applying as a (home student) Oxford undergrad. Didn't receive the offer until super late...!
Degree: PhD
Citizenship: Other
GPA: Not reported
GRE Quant: Not reported
GRE Verbal: Not reported

GRE AW: Not reported

Training Configuration

Training set size: 31411

Test set size: 7853

Training set class balance: Accepted=14074 (44.8%), Rejected=17337 (55.2%)

Test set class balance: Accepted=3519 (44.8%), Rejected=4334 (55.2%)

Train/test separation matters because it is the only way to estimate how the model will perform on data it has never seen. A model can always fit its training data well by memorizing patterns specific to those examples. The test set is what tells us whether it has learned something that generalizes, versus just memorized entries. This is critical when the trained model will be deployed on a public-facing webpage ('Will You Get In?'). Every real user who submits via the form is, by definition, a case the model has never seen during training. If we evaluated only on training data, we would have no honest way to know how the model will perform for live users. We would be reporting how well it memorized the past, not how reliable its predictions will be for someone using the site tomorrow.

Model name:	distilbert-base-uncased
Tokenizer name:	distilbert-base-uncased (matching pretrained tokenizer)
Max sequence length:	256
Batch size:	16
Epochs:	3
Learning rate:	2e-05
Optimizer:	torch.optim.AdamW
Device:	cuda

Model / Tokenizer Choice

DistilBERT (distilbert-base-uncased) was chosen over full BERT/RoBERTa/ALBERT because it's a distilled version of BERT: ~40% fewer parameters and ~60% faster at inference/training, while retaining roughly 97% of BERT's language understanding on downstream tasks. For a single binary classification head on top of relatively short, template-structured admissions text (not open-ended long-form prose), that tradeoff is very favorable. We don't need BERT's full capacity to separate "Accepted" vs "Rejected" from this input, and the smaller model makes fine-tuning on ~31K examples practical on ordinary (non-multi-GPU) hardware (particularly in my case, with a relatively low-end GPU). Its tokenizer is the matching pretrained WordPiece tokenizer (uncased, since admissions text has no case-sensitive signal we care about), which we load directly via AutoTokenizer / AutoModelForSequenceClassification so the tokenizer and model vocabulary stay in sync automatically.

Representative Output Log

```
epoch 1/3  step 50/1964  loss=0.6657
epoch 1/3  step 100/1964  loss=0.8100
...
epoch 1/3  step 1950/1964  loss=0.3126
epoch 1/3  step 1964/1964  loss=0.8299
```

```
epoch 1/3 complete -- avg train loss=0.5334, test accuracy=0.7702
epoch 2/3  step 50/1964  loss=0.5450
epoch 2/3  step 100/1964  loss=0.3477
...
epoch 2/3  step 1950/1964  loss=0.5244
epoch 2/3  step 1964/1964  loss=0.6870
epoch 2/3 complete -- avg train loss=0.4509, test accuracy=0.7882
epoch 3/3  step 50/1964  loss=0.3887
epoch 3/3  step 100/1964  loss=0.4568
...
epoch 3/3  step 1950/1964  loss=0.4986
epoch 3/3  step 1964/1964  loss=0.4785
epoch 3/3 complete -- avg train loss=0.4035, test accuracy=0.7852
```

Evaluation Metrics

Metrics (positive class = Accepted):

Accuracy: 0.7852

Precision: 0.7738

Recall: 0.7357

F1 score: 0.7543

Confusion matrix (rows = actual, columns = predicted):

	Prediction Accepted	Prediction Rejected
Actual Accepted	2589	930
Actual Rejected	757	3577

Class distribution:

- Actual: Accepted=3519 (44.8%), Rejected=4334 (55.2%)
- Predicted: Accepted=3346 (42.6%), Rejected=4507 (57.4%)

Probability examples (first 5 test rows):

[0] actual=Rejected predicted=Rejected P(Accepted)=0.185

[1] actual=Accepted predicted=Rejected P(Accepted)=0.158

[2] actual=Accepted predicted=Accepted P(Accepted)=0.852

[3] actual=Accepted predicted=Accepted P(Accepted)=0.978

[4] actual=Accepted predicted=Accepted P(Accepted)=0.989

Correctly classified examples:

[0] actual=Rejected, predicted=Rejected, P(Accepted)=0.031

Program: Political Science

University: New York University

Comments: Email from DGS. Got a [redacted] in mid-Feb, and informed by the DGS through email that consider the yield this year, they will not move me off the list.

Degree: PhD

Citizenship: International

GPA: Not reported

GRE Quant: Not reported

GRE Verbal: Not reported

GRE AW: Not reported

[2] actual=Accepted, predicted=Accepted, P(Accepted)=0.877

Program: Speech-Language Pathology

University: Boston University

Comments: Unknown

Degree: Master's

Citizenship: American

GPA: 3.96
GRE Quant: Not reported
GRE Verbal: Not reported
GRE AW: Not reported

[3] actual=Accepted, predicted=Accepted, $P(\text{Accepted})=0.989$

Program: Economics
University: Northwestern University

Comments: [redacted] without first year funding (no stipend; must pay tuition). Guaranteed 2nd year onward w funding as PhD

Degree: PhD
Citizenship: American
GPA: 3.98
GRE Quant: Not reported
GRE Verbal: Not reported
GRE AW: Not reported

Incorrectly classified examples:

[1] actual=Accepted, predicted=Rejected, $P(\text{Accepted})=0.165$

Program: English
University: University of California, Berkeley
Comments: Unknown
Degree: PhD
Citizenship: American
GPA: 3.8
GRE Quant: Not reported
GRE Verbal: Not reported
GRE AW: Not reported

[6] actual=Accepted, predicted=Rejected, $P(\text{Accepted})=0.474$

Program: Creative Writing Fiction
University: University of Houston
Comments: Unknown
Degree: PhD
Citizenship: American
GPA: 3.67
GRE Quant: Not reported
GRE Verbal: Not reported
GRE AW: Not reported

[9] actual=Rejected, predicted=Accepted, $P(\text{Accepted})=0.840$

Program: Political Science

University: University of Western Ontario
Comments: Unknown
Degree: Master's
Citizenship: International
GPA: 3.8
GRE Quant: Not reported
GRE Verbal: Not reported
GRE AW: Not reported

Interpretation

Bias: The model recalls Rejected applicants (85.6%) better than Accepted applicants (71.2%), predicting Accepted 3131 times vs. 3519 actual Accepted cases in the test set. This suggests some bias toward predicting the majority class (Rejected).

Vs. random/majority guessing: The majority-class baseline (always predicting the more common outcome in the test set) would score 55.2% accuracy. The fine-tuned model scores 79.1%, which is meaningfully above this baseline, indicating it has learned real signal beyond guessing the majority class.

Vs. earlier two-layer Neural Network: The two-layer NumPy neural network scored 70.0% test accuracy; the fine-tuned transformer scores 79.1%, an improvement, consistent with the transformer being able to use free-text fields (Comments, Program, University) that the purely numeric neural network could not.

Dataset sufficiency: Only about 20% of GPA/GRE fields are populated on average in this test set (many applicants do not report them), and labels come from self-reported, unverified GradCafe outcomes with no information on program competitiveness, applicant pool size, or redacted comment content (~12% of comments needed outcome-language redaction, per Step 2). That's enough signal to beat a naive majority-class guess, but it falls well short of what a real admissions office would need: no verified transcripts, no letters of recommendation, no interview data, and no guarantee the sample is representative of any single program's applicant pool. This model is a reasonable course-project demonstration of text+tabular fine-tuning, not a realistic admissions predictor.

Comparison to the Earlier Neural Network

On a basic level, the PyTorch model allows us to use non-numeric data (or data that can't be easily converted to a numeric representation) in weighing the prediction. The accuracy moved up to just shy of 80% from 70% with the switch from the simple neural net, so numerically speaking it appears the inclusion of the other fields was helpful. It is hard to tell specifically, because of the black box nature of the model, how much affect the inclusion of the new fields had on the weights. This does lead me say that the model "feels" more solid, but that is a perception based judgment that can never be backed by numbers. It is definitely more flexible, since it has the capability to examine data individually that the neural net could not (I can do a prediction on just the university and program for example). The model doesn't need all of the data to make a guess (no back-filled mean data that might not be an actual representation of the true mean). Since this is mostly just a toy model, there is an identifiable increase over the neural net, and it gives wider freedom in the types of data to consider, it seems justified.

Limitations and Ethics

This modeling path, of fine-tuning a pre-trained language model, definitely saves time, since the base details are already decided for you. It also provides the linguistic back-end to make the more free entry text fields usable. Being able to stack the textual entries with the structured data helps to (hopefully) fine tune the purely numeric with some of the potentially more “soft” scoring items, comments that could include statements about interviews, research completed, or relevant work experience. However, it is in no way a perfect method. Self-reported data is inherently biased. Someone is not going to report that they did poorly on an interview or that the quality of their admissions essay was lack luster. They may even lie about scores or omit them so a low GRE score does not make them look bad. Because this is a major limitation of the data, it becomes a major limitation of the model. Garbage in, garbage out.

Would I ever recommend this as a real decision making tool? Never. Some people might find it useful as a “mostly accurate” representation, **if** they were honest about the data they gave, and **if** the underlying self reported data was verifiably accurate. But those things can never be guaranteed in this context. As a learning experience, this project teaches a lot about ML as well as data hygiene. I would still leave this soundly in the “fun” level of interest. Anyone representing the results beyond that is being dishonest with the user. The easily visible and understood disclaimer is a step in the right direction. There could be presentational design aspects that also communicate the “just-for-fun” aspect, such as cartoonish representations of varying scores.